바이브코딩으로 게임 만들기

웹 개발자의 과일 게임 개발 여정기

임근석 쓸모랩 대표

2026년 4월 17일 | 게임 개발자를 위한 Claude Code 꿀팁 | 40분

도입

01 도입 — 자기소개와 바이브코딩
02 도구 — Claude Code와 에이전트 시스템
03 여정 — 10개 세션의 게임 진화
04 디버깅 — 실전 버그 수정 사례
05 리디자인 — 아케이드 테마 변신
06 꿀팁 — 게임 개발자를 위한 6가지

임근석

- 쓸모랩 대표
- 웹 풀스택 개발자
- 게임 개발 경험: 없음
- AI 도구 매니아
- Claude Code 헤비 유저

```
{
  "name": "임근석",
  "role": "CEO",
  "company": "쓸모랩",
  "stack": "Web FS",
  "game_exp": null
}
```

게임 개발을 해본 적 없는
웹 개발자가
게임을 만들 수 있을까?

VIBE CODING

"Just describe what you want and let AI build it"

IDEA
DIRECTION
FEEDBACK

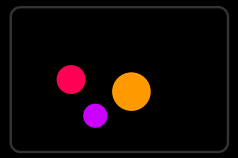
prompt

CLAUDE
CODE

PLAN
IMPLEMENT
DEBUG
TEST

output

WORKING
GAME



Physics
Mobile
Arcade UI

과일 게임 (Suika Game)

- 일본에서 시작된 물리 퍼즐 게임
- 같은 과일끼리 합치면 진화
- 체리 → 딸기 → ... → 수박
- **물리 엔진**이 핵심 게임플레이

왜 이것을?

- 유튜브에서 '엑셀로 수박 게임' 영상 발견
- '나도 웹으로 만들 수 있지 않을까?'
- 물리 엔진 = **새로운 도전**

22개 커밋, 2일, 10세션

- 물리 기반 과일 합치기 게임
- 아케이드 스타일 픽셀 아트
- 네온 글로우 UI
- 데스크탑 + 모바일 대응
- **코드 한 줄도 직접 안 짬**

Vanilla JS ⟶ TypeScript + React
⟶ Cannon.js ⟶ Arcade Redesign
⟶ Pixel Art

10 Sessions, 2 Days, 1 Game

S1	Vanilla JS + Physics Engine + 51 Tests	3/29
S2	Migration Design (TS + React + Cannon.js)	3/29
S3	TypeScript + React + Cannon.js Implementation	3/29
S4	Code Review + Validation Fixes	3/29
S5	Playwright Test + Space Bug + Gemini SVG Design	3/29
	--- Day 2 ---	
S6	Collision Bug Fix + Mobile Optimization	3/30
S7	Jest Cleanup + Mouse Click Support	3/30
S8	Arcade Redesign Brainstorming + Planning	3/30
S9	Arcade Redesign Implementation (5 commits)	3/30
S10	Pixel Art Fruits + Sync Rendering Fix	3/30

22 commits | 7 bug fixes | 3 major rewrites | 0 lines of code written by hand

도구: Claude Code 생태계

CLI 기반 AI 코딩 도구

- 터미널에서 바로 실행
- 파일 읽기/쓰기/편집
- 셸 명령 실행
- Git 연동
- **대화형 개발**

```
$ claude
> 과일 게임을 만들어줘
```

핵심 특징

컨텍스트	프로젝트 전체 이해
도구	Read, Write, Bash, Grep...
에이전트	서브에이전트 위임
스킬	재사용 가능한 워크플로우
후크	이벤트 기반 자동화

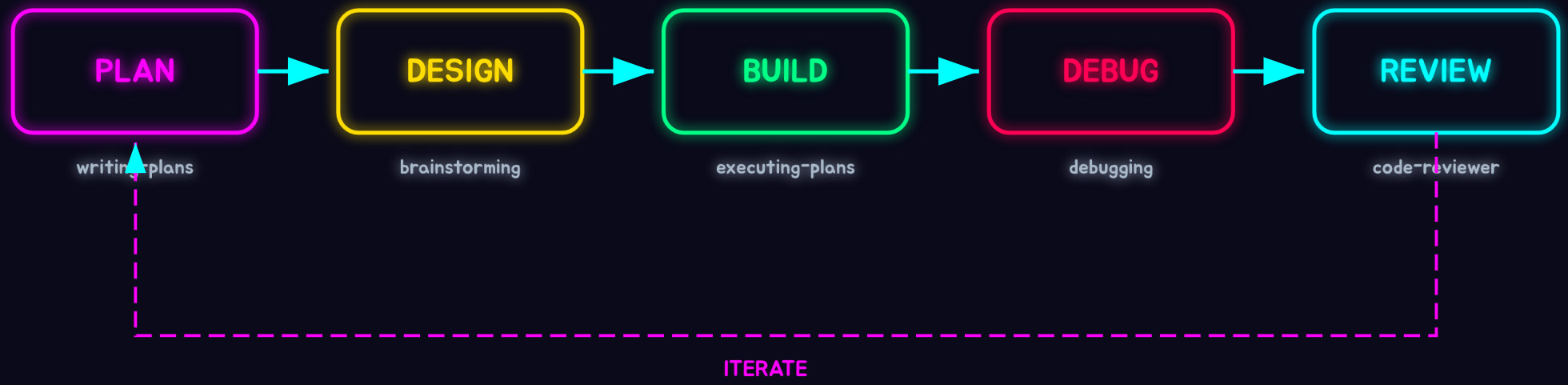
개발 워크플로우 프레임워크

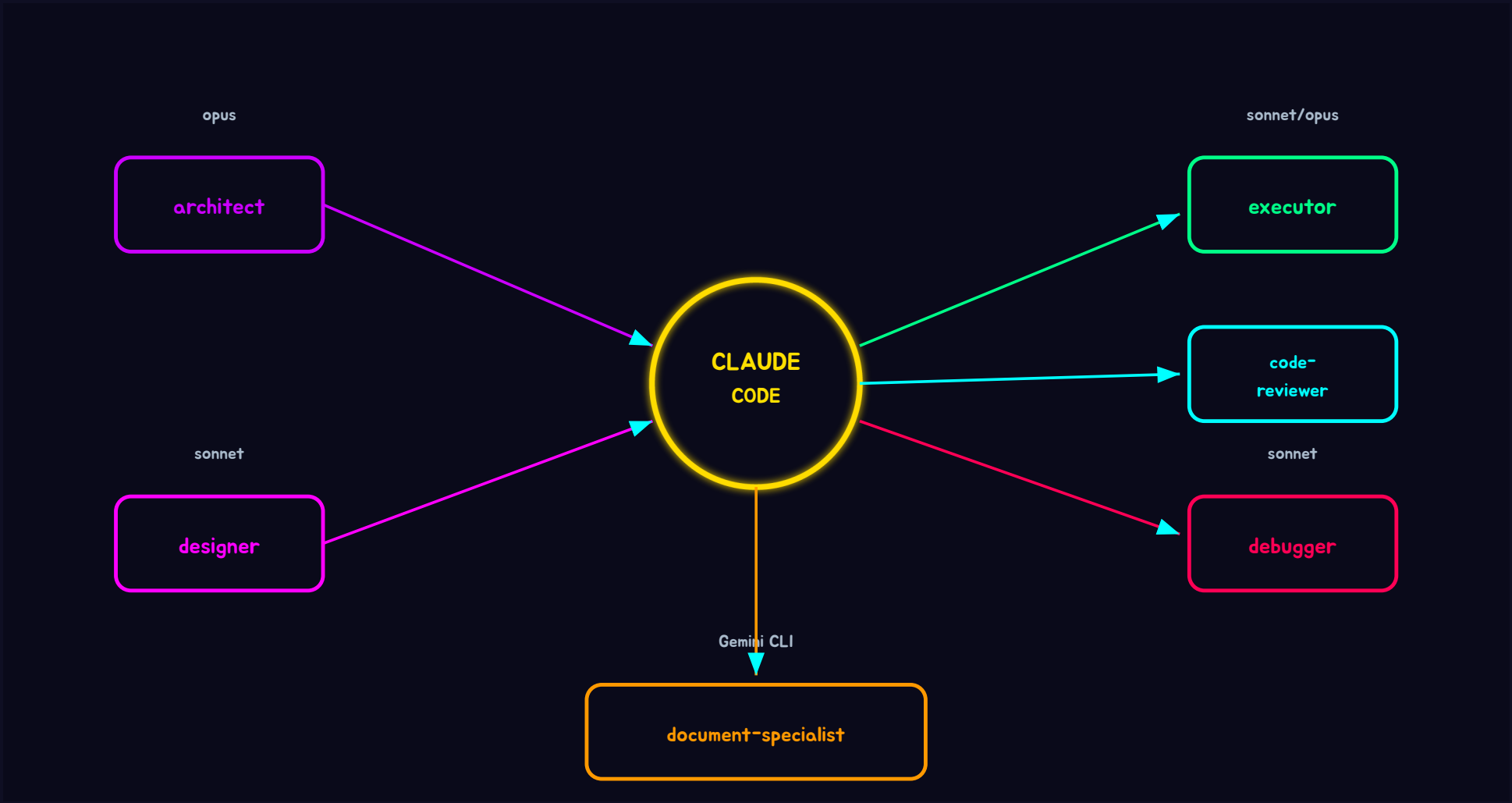
Claude Code 위에 올라가는 **구조화된 개발 프로세스**

- 브레인스토밍 → 설계 → 구현 → 검증
- 각 단계에 전문 스킬 매핑
- 서브에이전트 자동 위임
- 코드 리뷰 + 검증 단계 내장

사용한 스킬 목록

brainstorming	아이디어 탐색
writing-plans	설계 문서 작성
executing-plans	계획 실행
subagent-driven	병렬 구현
debugging	버그 추적
code-review	코드 검증
playwright-cli	브라우저 테스트


PLAN
writing-plans
DESIGN
brainstorming
BUILD
executing-plans
DEBUG
debugging
REVIEW
code-reviewer
ITERATE


opus
architect
sonnet
designer
CLAUDE
CODE
sonnet/opus
executor
code-
reviewer
sonnet
debugger
Gemini CLI
document-specialist

설계 에이전트

architect	아키텍처 설계 (Opus)
designer	UI/UX 디자인 제안
planner	실행 계획 수립

실행 에이전트

executor	코드 구현
debugger	버그 원인 추적
code-reviewer	코드 품질 검증

특수 에이전트

doc-specialist	외부 문서 조회
qa-tester	테스트 실행
git-master	Git 커밋 관리

외부 AI 연동

Gemini CLI SVG 아트 디자인 생성

```
omc ask gemini "11개 과일을 SVG로 디자인해줘"
```

Claude + Gemini

각 AI의 강점을 활용:

- **Claude Code**: 코드 생성, 디버깅, 아키텍처
- **Gemini**: 시각 디자인, SVG 아트

실제 사용 예시

```
omc ask gemini \
  "11개 과일을 과일 게임
   스타일로 SVG 디자인해줘"
```

→ 결과를 Claude가 코드에 통합

```
      Claude Code
  │ "Gemini한테 디자인 맡겨"
          │
      Gemini CLI
    │ → 11개 SVG 생성
          │
      Claude Code
  │ ← FruitSVG.ts에 통합
    │ ← 렌더링 코드 수정
    │ ← 캐시 최적화 추가
          Done!
```

한 줄 프롬프트 → 기획서 → 설계 → 구현 → 리뷰

모든 단계를 AI가 수행

개발자는 방향만 제시

여정: 10개 세션의 게임 진화

프롬프트

'엑셀로 만든 수박 게임을 웹 Canvas + JavaScript로 구현해줘.
물리 엔진, 충돌 감지, 과일 합치기 메커니즘 포함.'

AI가 한 일

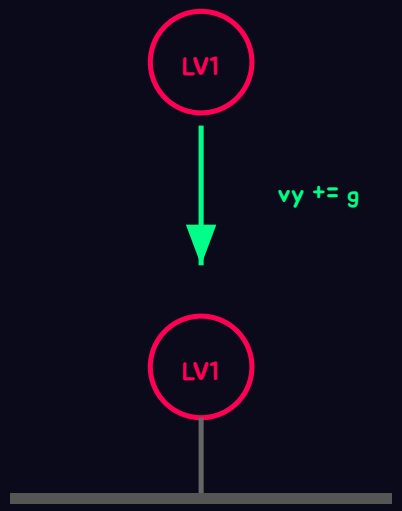
1. 기획서 작성 (writing-plans)
2. 6개 모듈 구현
3. 51개 단위 테스트 작성
4. TDD 기반 개발 (ralplan)

생성된 파일 구조

```
src/
├── Game.js
├── Fruit.js
├── Physics.js
├── CollisionDetector.js
├── GameState.js
└── InputHandler.js
tests/
└── 51 unit tests ✓
```

Physics Engine Concepts

GRAVITY



vx *= 0.75 (friction)

COLLISION

d = sqrt(dx2+dy2)

d < r1 + r2 ?

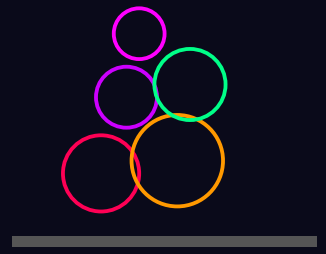
MERGE!

GAME OVER

SPAWN LINE

fruit.y < line

for 3+ seconds



GAME OVER

ralplan 스킬

RALPLAN = Research → Analyze → Locate → Plan

1. 요구사항 분석
2. 코드 탐색
3. 태스크 분할
4. TDD 순서 실행

테스트 커버리지

- `Fruit.test.js` — 생성, 레벨, 크기
- `Physics.test.js` — 중력, 마찰, 반발
- `CollisionDetector.test.js` — 충돌, 합치기
- `GameState.test.js` — 점수, 게임오버

코드 예시 (Physics.js)

```
// 오일러 적분 기반 중력
fruit.vy += this.gravity;

// 수평 마찰만 적용
fruit.vx *= 0.75;

// 벡터 반발력 (법선 방향)
const nx = dx / dist;
const ny = dy / dist;
const impulse = -(1 + restitution)
  * relVel / 2;
fruit1.vx += impulse * nx;
fruit1.vy += impulse * ny;
```

세션 2: 설계 문서

'TypeScript + React + Cannon.js로
마이그레이션 해줘. 먼저 설계 문서를 작성해.'

architect 서브에이전트가 설계:

- Vite 빌드 도구
- Cannon-es 물리 엔진
- React 컴포넌트 구조
- TypeScript 타입 정의

세션 3: 구현

executing-plans 스킬로 실행:

Game.tsx	게임 루프 + Canvas
GameCanvas.tsx	Canvas 엘리먼트
GameUI.tsx	점수판 + UI
Fruit.ts	과일 모델
CannonPhysics.ts	물리 래퍼
CollisionHandler.ts	충돌 + 합치기
GameState.ts	상태 관리
FruitSVG.ts	과일 그래픽

Technology Evolution

프롬프트

'코드 리뷰를 해줘.'

code-reviewer 발견 사항

- `InputHandler.destroy()` 누락
- 이벤트 리스너 메모리 누수
- SVG 로딩 에러 핸들링 부재
- 타입 안전성 개선 필요

수정 결과

```
// cleanup 추가
useEffect(() => {
  const handler = new InputHandler();
  // ...
  return () => {
    handler.destroy(); // 리스너 정리
    physics.cleanup(); // 물리 정리
  };
}, []);
```

자동 리뷰 → 자동 수정 → 커밋

프롬프트

'분명 아까는 잘 동작했는데
이제는 스페이스 누르면 동작도 안하네?'

playwright-cli로 테스트

브라우저 자동화로 버그 재현 확인

systematic-debugging 스킬

원인 추적 → React StrictMode 발견

원인

React StrictMode가 `useEffect`를 **2번 실행**

1. 1차 실행: InputHandler 생성
2. 1차 cleanup: **destroy() 호출**
3. 2차 실행: 같은 ref 재사용
4. 결과: **리스너 없는 상태**

↳ 스페이스 키가 아무 반응 없음

React StrictMode Bug

BUG (useRef)

1. useEffect runs (1st)
2. InputHandler created
3. cleanup: destroy()!
4. useEffect runs (2nd)
5. Same ref, no listeners!

   Space key does nothing

FIX (new in useEffect)

1. useEffect runs (1st)
2. new InputHandler()
3. cleanup: destroy()
4. useEffect runs (2nd)
5. NEW instance + listeners!

   Space key works!

프롬프트

'gemini-3.1-pro-preview로
gemini cli를 제어해서 과일들을
디자인해서 반영해줘.'

과정

1. `omc ask gemini` 호출
2. 11개 과일 SVG 생성 요청
3. JSON으로 결과 수신
4. Claude가 FruitSVG.ts에 통합

결과: 11개 과일 SVG

LV1 체리	LV2 딸기	LV3 포도
LV4 데코폰	LV5 감	LV6 사과
LV7 배	LV8 복숭아	LV9 파인애플
LV10 멜론	LV11 수박	

각 과일: 하이라이트 + 줄기/잎 + 표정 포함
viewBox='0 0 100 100' SVG

자연어로 계속 개선

프롬프트	문제	해결
'왜 시작할 때 공은 과일로n안되어 있어?'	미리보기가 단색 원	FruitSVG 사용으로 변경
'다 좋은데 스폰 과일에n둥근 원은 왜 없어?'	`ctx.stroke()` 누락	테두리 코드 추가
'왜 스페이스 누르고 나면n현재 커서가 안보여?'	쿨다운 중 프리뷰 숨김	투명도 0.3으로 표시

일상적인 대화 = 버그 리포트 = 프롬프트

확장: 모바일과 인터랙션

프롬프트

'과일들이 합쳐질 때
가끔 하나가 안 사라져.'

원인

CollisionHandler에서 과일 제거 시
Cannon.js 물리 바디가 남아있음

교훈

추상화 레이어 사이의 불일치
→ AI가 빠르게 추적 가능

수정

```
// Before: 데이터만 제거
collisionHandler.removeFruit(fruitA);
// → 물리 바디 잔존!

// After: 물리 + 데이터 동시 제거
physics.removeFruit(fruitA);
// → 바디도 함께 제거
```

사용 도구

- playwright-cli — 버그 재현
- systematic-debugging — 원인 추적
- debugger 서브에이전트

프롬프트

'모바일에서도 동작하게 해줘.
그리고 파비콘도 넣어.'

구현 항목

- 터치 이벤트 (start/move/end)
- CSS 스케일 좌표 변환
- iOS 바운스 방지
- 뷰포트 고정
- 체리 SVG 파비콘

터치 → 캔버스 좌표 변환

```
canvas.addEventListener('touchmove',
  (e) => {
    const rect =
      canvas.getBoundingClientRect();
    const scaleX =
      canvas.width / rect.width;
    const touch = e.touches[0];
    this.mouseX =
      (touch.clientX - rect.left)
      * scaleX;
  }
);
```

CSS transform 스케일을 역변환하여
터치 좌표를 캔버스 좌표계로 매핑

Jest 테스트 제거

Cannon.js 도입 후 자체 물리 엔진의
단위 테스트가 무의미해짐

- `jest.config` 삭제
- `tests/` 삭제
- jest 관련 의존성 제거

기술 부채 = AI에게도 정리 요청 가능

마우스 클릭 지원

```
// mousemove → 위치 추적
canvas.addEventListener(
  'mousemove', (e) => {
    this.mouseX = e.offsetX;
  }
);

// click → 과일 드롭
canvas.addEventListener(
  'mousedown', () => {
    this.dropRequested = true;
  }
);
```

세션	입력 방식	추가 사항
세션 1	키보드 (←→ + Space)	기본 조작
세션 5	1. `preventDefault`	스크롤 방지
세션 6	1. 터치 (모바일)	좌표 변환
세션 7	1. 마우스 클릭	3가지 입력 통합

리디자인: 아케이드 테마 변신

'이 게임을 팩맨 같은
오락실 느낌으로 바꾸고 싶어'

brainstorming 스킬

디자인 방향을 체계적으로 탐색:

1. 시각 스타일 후보 제안
2. 각 방향의 장단점 분석
3. 기술 제약 사항 확인
4. 최종 방향 결정

결정 사항

- 팩맨/갤러그 아케이드 감성
- 검정 배경 + 네온 하이라이트
- Press Start 2P 폰트
- 픽셀 아트 과일

서브에이전트 역할

architect (Opus)
- 전체 리디자인 범위 설정
- 게임 로직 건드리지 않는 전략

designer (Sonnet)
- 컬러 팔레트 제안
- UI 레이아웃 설계
- 아케이드 참고 사례 조사

brainstorming → writing-plans
두 스킬을 순차적으로 사용

writing-plans 스킬 결과

2개 문서 자동 생성:

디자인 스펙 (24KB)
- 8색 컬러 팔레트
- 타이포그래피 규칙
- UI 레이아웃 (ASCII)
- Canvas 렌더링 변경
- 11개 과일 픽셀 아트 명세

실행 계획 (12KB)
- 6개 파일 변경 계획
- 7개 태스크 분할
- 각 태스크별 코드 스니펫

변경 대상

index.html	폰트 링크
index.css	배경색 + 폰트
GameCanvas.tsx	네온 테두리
GameUI.tsx	전면 재작성
Game.tsx	Canvas 렌더링
FruitSVG.ts	픽셀 아트 교체

게임 엔진 파일은 일절 변경 없음
(Physics, Collision, Input)

Arcade Color Palette

Background	Canvas	Title	Neon Cyan	Neon Pink	Spawn Red	Score
#0a0a1a	#000000	#FFE000	#00ffff	#ff00ff	#ff0055	#ffffff

FRUIT GAME

1UP	HI SCORE	LEVEL
08640	99999	01

Press Start 2P font family

executing-plans + subagent-driven-development

설계 문서의 7개 태스크를 순서대로 자동 실행:

Task	내용	커밋
1	Press Start 2P 폰트 + 배경 설정	3120015
2	캔버스 네온 시안 테두리 + 글로우	16ce676
3	1UP / HI SCORE / LEVEL UI 재작성	c700c7e
4	Canvas 배경 검정 + 네온 스폰라인	ac3c192
5	네온 글로우 과일 + 게임오버 화면	9e8f1f9

Press Start 2P

8비트 픽셀 폰트로 전체 UI 통일

```
<link href="https://fonts.
  googleapis.com/css2?family=
  Press+Start+2P" rel="stylesheet">
```

배경색

```
body {
  background: #0a0a1a;
  font-family:
    'Press Start 2P', monospace;
}
```

Before → After

GameUI.tsx 전면 재작성

```
// 아케이드 3열 점수판
<div className="score-panel">
  <div>
    <span className="label">1UP</span>
    <span className="score">
      {score.toString()
            .padStart(5, '0')}
    </span>
  </div>
  <div>
    <span className="label">
      HI SCORE
    </span>
    <span className="score">
      {hiScore}
    </span>
  </div>
</div>
```

레이아웃

★ FRUIT GAME ★

1UP	HI SCORE	LEVEL
08640	99999	01

← → SPACE | CLICK

과일 렌더링

```
// 네온 글로우 효과
ctx.shadowColor =
  NEON_COLORS[fruit.level];
ctx.shadowBlur = 8;

// 레벨별 네온 색상
const NEON_COLORS = {
  1: '#ff0055',  // 체리
  2: '#ff3366',  // 딸기
  3: '#cc00ff',  // 포도
  // ...
  11: '#ff1155', // 수박
};
```

게임오버 화면

```
// 아케이드 게임오버
ctx.fillStyle =
  'rgba(0, 0, 0, 0.88)';
ctx.fillRect(0, 0, W, H);

ctx.font =
  '32px "Press Start 2P"';
ctx.fillStyle = '#FFE000';
ctx.fillText('GAME OVER', ...);

// 깜박이는 PRESS SPACE
if (Math.floor(Date.now()
    / 500) % 2 === 0) {
  ctx.fillStyle = '#ff00ff';
  ctx.fillText(
    'PRESS SPACE', ...);
}
```

프롬프트

'과일들도 도트 느낌의 픽셀 아트로 바꿔줘.'

Pixel Art Fruits (11 Levels)

LV1 Cherry · LV2 Strawberry · LV3 Grape · LV4 Dekopon · LV5 Persimmon · LV6 Apple · LV7 Pear · LV8 Peach · LV9 Pineapple · LV10 Melon

LV11 Watermelon

All fruits rendered with SVG <rect> based pixel art + imageSmoothingEnabled=false

SVG rect 기반 도트 그래픽

```svg
<svg viewBox="0 0 12 12">
  <!-- 1px = 1 rect -->
  <rect x="5" y="0"
    width="1" height="1"
    fill="#663300"/>
  <rect x="4" y="1"
    width="1" height="1"
    fill="#ff0055"/>
  <rect x="5" y="1"
    width="1" height="1"
    fill="#ff3377"/>
  <!-- ... -->
</svg>
```

viewBox='0 0 12 12' (LV11만 16×16)

렌더링 설정

```
// 픽셀 크리스프 보장
ctx.imageSmoothingEnabled = false;
```

Before → After

Gemini SVG

→

Pixel Art

디버깅: AI와 함께 버그 잡기

SVG Async Race Condition

Image.onload (async)

img.src = data:

FRAME 1

onload fires

canvas empty!

FRAME 2

now visible

data URI != synchronous

Even data: URIs go through
the async image decode pipeline

fillRect (sync)

ctx.fillRect()

FRAME 1

visible!

fillRect = instant

Pixel art = rectangles
No Image object needed

systematic-debugging 스킬

1. **증상 수집**
   '첫 프레임에 과일이 안 보여'
2. **가설 수립**
   - SVG 로딩 실패?
   - Canvas 초기화 순서?
   - 비동기 타이밍?
3. **증거 수집**
   debugger 서브에이전트가
   코드 추적
4. **근본 원인 확인**
   `Image.onload`는 data URI도 비동기

수정 전략

발견: 픽셀 아트는 `<rect>`로만 구성

→ `Image` 객체가 필요 없음!

```
// Before: 비동기
const img = new Image();
img.onload = () => {
  ctx.drawImage(img, 0, 0);
};
img.src = 'data:image/svg+xml;...';

// After: 동기
ctx.fillStyle = color;
ctx.fillRect(x, y, 1, 1);
```

비동기 → 동기 = 레이스 컨디션 원천 제거

버그	원인	수정
Space 키 스크롤	`preventDefault` 누락	InputHandler에 추가
Space 키 미동작	StrictMode 이중 실행	useEffect 내 생성
미리보기 단색 원	FruitSVG 미사용	`getCanvasForLevel()`
테두리 없음	`ctx.stroke()` 누락	코드 추가
발사 후 커서 사라짐	쿨다운 시 숨김	투명도 0.3
과일 미제거	레이어 불일치	`physics.removeFruit()`
첫 프레임 빈 화면	async Image	fillRect 동기

AI는 '왜 안 돼?'라는 질문에
코드 전체 컨텍스트를 보고 답한다

사람: 증상 설명

AI: 원인 추적 + 수정 + 커밋

7개 버그를 모두 자연어 대화로 수정

Claude Code 꿀팁

writing-plans 스킬

코드 작성 전에 설계 문서를 먼저 생성

- 파일 구조 결정
- 모듈 간 의존성 정의
- 태스크 분할
- 각 태스크별 코드 스니펫

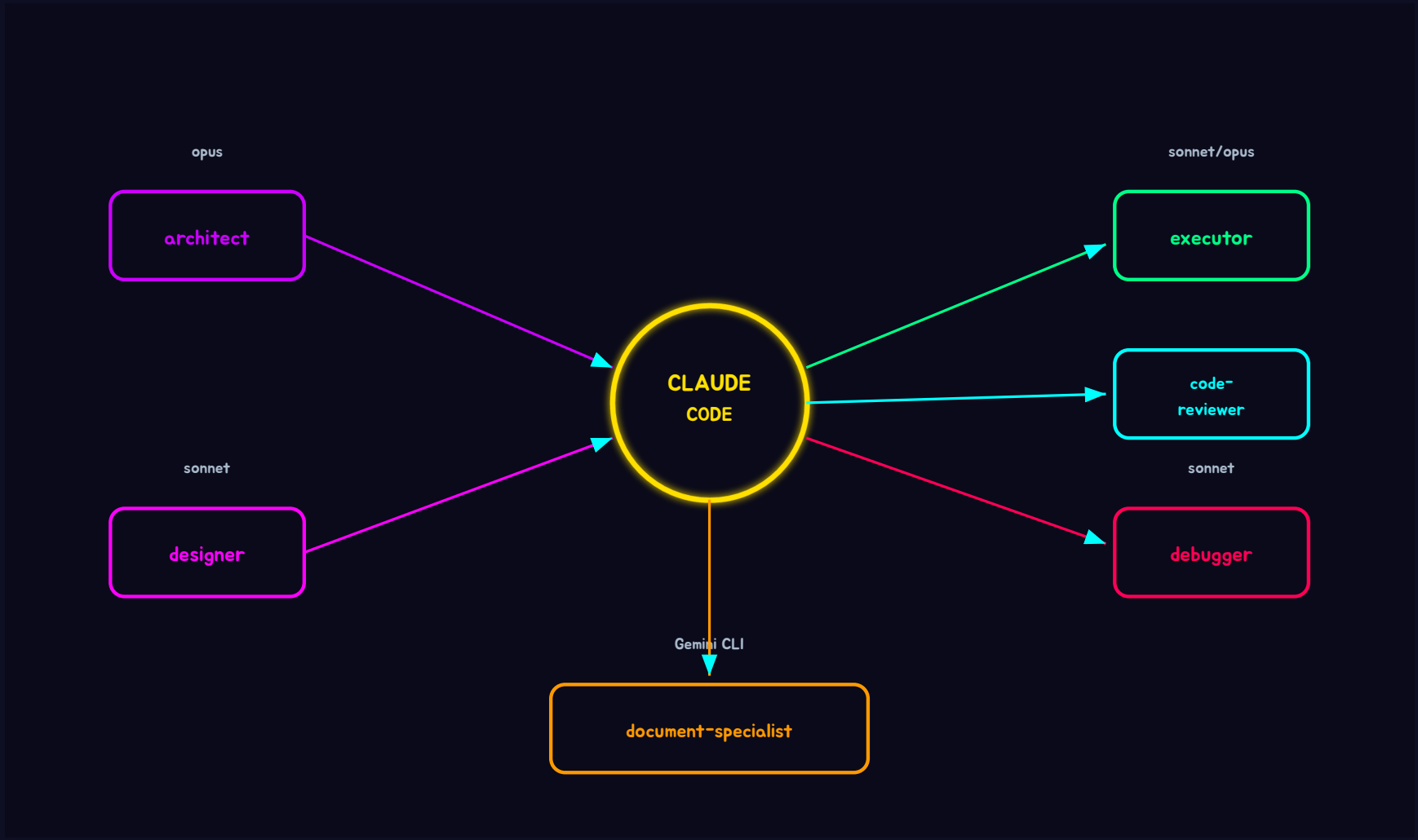
'설계 문서를 먼저 작성해줘' 한 문장이면 충분합니다.

왜 중요한가?

- AI도 구조 없이 코드를 쓰면 혼란
- 설계 문서 = AI의 실행 맵
- 대규모 변경도 안전하게 진행
- 리디자인 시 영향 범위 명확화

실제 결과

아케이드 리디자인:
24KB 스펙 + 12KB 실행계획
→ 5커밋으로 무결하게 완료


opus
architect
sonnet
designer
CLAUDE
CODE
sonnet/opus
executor
code-
reviewer
sonnet
debugger
Gemini CLI
document-specialist

설계는 architect, 구현은 executor, 리뷰는 code-reviewer

03. 증거 기반 디버깅

`systematic-debugging` 스킬은:

- 가설을 세우고
- 증거를 수집하고
- 근본 원인을 찾는다

'왜 안 돼?' → AI가 코드 전체를 보고
React StrictMode까지 추적

04. 멀티 AI 협업

각 AI의 강점을 활용:

- **Claude**: 코드, 아키텍처, 디버깅
- **Gemini**: 시각 디자인, SVG 아트
- **Codex**: 대안 코드 생성

```
# Gemini에게 디자인 요청
omc ask gemini \
  "11개 과일 SVG 디자인해줘"
```

05. 자연어로 반복 개선

프롬프트 = 일상 대화:

- '왜 시작할 때 공은 과일로 안 돼있어?'
- '다 좋은데 둥근 원은 왜 없어?'
- '스페이스 누르면 동작 안 해'

팀원에게 말하듯이
AI에게 이야기하세요

06. 모든 것을 리뷰하라

AI가 작성한 코드도 **검증 필수**:

- `code-reviewer` 서브에이전트
- `requesting-code-review` 스킬
- `playwright-cli`로 실제 테스트

실제 효과

세션 4 코드 리뷰에서 발견:

- 메모리 누수 (`destroy()` 누락)
- 에러 핸들링 부재
- 타입 안전성 문제

Claude Code Tips for Game Devs

01 Plan Before You Build

writing-plans creates
step-by-step execution plans

02 Delegate to Agents

executor, architect, designer
work in parallel contexts

03 Debug with Evidence

systematic-debugging traces
root cause, not symptoms

04 Multi-AI Collaboration

Gemini for art, Claude for code
each AI's strength matters

05 Iterate Naturally

Talk to AI like a teammate
"Why isn't this working?"

06 Review Everything

code-reviewer catches
what you might miss

You don't need to know game dev.

You need to know how to collaborate with AI.

숫자로 보는 여정

총 세션	10개
총 커밋	22개
개발 기간	2일
수정한 버그	7개
주요 리팩터링	3회
직접 작성한 코드	0줄
사용한 AI 모델	3개
사용한 서브에이전트	6종

핵심 교훈

1. 도메인 지식 < 협업 능력
게임 개발을 몰라도 됩니다.
AI와 소통하는 법을 알면 됩니다.

2. 프롬프트 = 대화
'왜 안 돼?'가 최고의 프롬프트입니다.

3. 구조가 품질을 결정
설계 → 구현 → 리뷰 사이클이
AI 코드의 품질을 보장합니다.

소스 코드

```
src/
├── components/
│   ├── Game.tsx        ← 게임 루프
│   ├── GameCanvas.tsx ← Canvas
│   └── GameUI.tsx      ← 아케이드 UI
├── engine/
│   ├── Fruit.ts        ← 과일 모델
│   ├── CannonPhysics.ts ← 물리
│   ├── CollisionHandler.ts
│   ├── GameState.ts
│   ├── InputHandler.ts
│   └── FruitSVG.ts    ← 픽셀아트
├── styles/index.css
├── App.tsx
└── index.tsx
```

변경 이력 요약

엔진 코드	세션 1-3에서 완성
UI 코드	세션 8-9에서 재작성
그래픽	세션 5, 10에서 2회 교체
입력	세션 1→5→6→7 점진 확장
테스트	세션 1 작성 → 세션 7 제거

아키텍처 원칙

- 엔진/렌더링/UI 분리
- 리디자인 시 엔진 무변경
- 입력 추상화로 멀티 플랫폼

지금 가능한 것

- 물리 기반 게임 2일 만에 완성
- 기획 → 구현 → 디자인 → 디버깅 전부 AI
- 모바일 + 데스크탑 동시 대응
- 아케이드 리디자인까지 자동

게임 개발 적용 포인트

- **프로토타이핑**: 아이디어 → 플레이 가능
- **에셋 생성**: SVG, 픽셀 아트, UI
- **디버깅**: 물리 버그, 렌더링 이슈
- **플랫폼 확장**: 웹 → 모바일

앞으로의 가능성

- 3D 게임 엔진 연동 (Three.js)
- 멀티플레이어 서버 구축
- 사운드 디자인 AI 연동
- CI/CD + 자동 배포

AI는 도구입니다.
여러분의 창의성이 게임을 만듭니다.

1단계: Claude Code 설치

```
npm install -g @anthropic-ai/
claude-code

claude
```

2단계: 첫 프롬프트

> 과일 게임을 만들어줘

이 한 줄이 모든 것의 시작이었습니다.

추천 워크플로우

1. 기획서부터 요청
2. 설계 문서 리뷰
3. 구현 실행
4. 브라우저 테스트
5. 자연어로 피드백
6. 반복

'만들어줘' 한 마디로 시작하세요.

Q & A

임근석 · 쓸모랩 대표

감사합니다!